



Universidad Nacional Experimental de Guayana

Vicerrectorado Académico

Coordinación General de Pregrado

P.F.G: Ingeniería en Informática

Unidad Curricular: Lenguaje y Compiladores

Tema 4:

Análisis Léxico

Profesor :

Félix Márquez

Alumnos:

Shirley Cedeño V-30.413.183 S2

Nelson Bueno V-27.407.263 S2

Angel Rodriguez V-30.437.205 S1

Rayc Yanez V-28.215.618 S2

Ciudad Guayana, julio del 2026

Resumen Académico

El presente informe documenta el diseño, la implementación y el análisis de analizadores léxicos, etapa que constituye la primera fase del proceso de compilación. Se aborda inicialmente la fundamentación teórica que delimita el alcance computacional de los Autómatas Finitos (AFD) frente a los Autómatas de Pila (AP) para el reconocimiento de lenguajes regulares y de libre contexto, respectivamente. En el ámbito práctico, se desarrollan dos analizadores léxicos operacionales: el primero construido algorítmicamente en Python utilizando expresiones regulares para la validación estructural de archivos de configuración de contenedores (Dockerfiles); el segundo, generado de manera automatizada en lenguaje C mediante el metacompilador FLEX, diseñado para procesar un subconjunto del lenguaje de programación Rust. Finalmente, se extrapolan los conceptos de la teoría de lenguajes formales al área de la seguridad informática, demostrando cómo la tokenización mediante FLEX optimiza la validación de reglas de detección de intrusos (Snort) en entornos críticos.

Introducción

El proceso de traducción de un lenguaje formal es una tarea computacional compleja estructurada en múltiples fases secuenciales. La primera y más crítica de estas etapas es el análisis léxico, cuyo propósito radica en transformar un flujo continuo de caracteres en unidades lógicas con significado semántico y sintáctico, denominadas tokens. El dominio riguroso de esta fase no solo viabiliza la construcción de compiladores e intérpretes, sino que garantiza la integridad estructural de la información antes de su procesamiento profundo.

En este contexto, el presente trabajo tiene como objetivo documentar el desarrollo práctico y el análisis teórico de mecanismos de evaluación léxica bajo dos enfoques metodológicos complementarios. En primer lugar, se explora la construcción manual de un motor léxico fundamentado en expresiones regulares para la validación de archivos Docker. Posteriormente, se aborda la automatización de este ciclo de desarrollo mediante el uso de metacompiladores, específicamente FLEX, aplicado al reconocimiento de estructuras del lenguaje Rust.

Asimismo, el documento establece las fronteras teóricas de estas herramientas al contrastarlas con modelos de mayor poder computacional, como los Autómatas de Pila, y proyecta la aplicabilidad de las técnicas de tokenización en la resolución de problemas reales del ámbito de la ciberseguridad, consolidando la relevancia de la teoría de lenguajes formales en la ingeniería de software moderna.

Autónoma de Pila

En el contexto del tema de Análisis Léxico, es imperativo establecer los límites computacionales de las herramientas utilizadas. Mientras que la fase léxica se resuelve íntegramente mediante Autómatas Finitos (AFD), la evaluación de la estructura gramatical del programa fuente requiere un salto en el poder de cómputo hacia los Autómatas de Pila.

Un Autómata de Pila (AP) es un modelo computacional teórico que amplía las capacidades de un autómata finito convencional mediante la incorporación de una memoria auxiliar estructurada como una pila (LIFO: Last In, First Out). Esta característica le permite almacenar y recuperar una cantidad ilimitada de información, lo cual es fundamental para procesar estructuras anidadas propias de las gramáticas de libre contexto (GLC).

Formalmente, un AP se define mediante la siguiente séptupla matemática:

$$M = (Q, \Sigma, \Gamma, \delta, q_0, Z_0, F)$$

Donde:

- **Q:** Conjunto finito de estados.
- **Σ :** Alfabeto finito de entrada (los caracteres o tokens leídos).
- **Γ :** Alfabeto finito de la pila (símbolos almacenables en la memoria auxiliar).
- **δ :** Función de transición que determina el siguiente estado y la operación a realizar en la pila (apilar, desapilar o mantener).
- **q_0 :** Estado inicial del que parte el autómata.

- **Z₀**: Símbolo inicial de la pila (utilizado para marcar el fondo de la memoria).
- **F**: Conjunto de estados de aceptación.

Igualmente se pueden explorar ejemplos prácticos de un Autómata de Pila

Ejemplo 1 - Reconocimiento de lenguajes simétricos y conteo

El caso fundamental que demuestra la necesidad irrefutable de un AP es el lenguaje formal

$L = \{a^n b^n \mid n \geq 0\}$, el cual exige que exista una secuencia de caracteres 'a' seguida por una cantidad estrictamente idéntica de caracteres 'b'.

- **Traza de ejecución:** El autómata inicia en un estado q_0 con un símbolo base Z_0 en su pila. Por cada 'a' que el cabezal lee de la cinta de entrada, el autómata ejecuta una transición que apila un símbolo **A**. Al detectar el primer carácter 'b', el autómata cambia a un estado q_1 . En este nuevo estado, por cada 'b' procesada, la función de transición dictamina que se debe desapilar (eliminar) un símbolo **A** del tope de la pila.
- **Condición de aceptación:** Si al terminar de leer la cadena completa, el autómata verifica que la pila está vacía (o solo contiene el símbolo inicial Z_0), la cadena es aceptada. Si faltan caracteres 'b' (quedan símbolos **A** en la pila) o sobran caracteres 'b' (se intenta desapilar una pila vacía), el autómata rechaza la cadena. Este mecanismo de almacenamiento dinámico dota a la máquina de la capacidad de "contar", una operación matemáticamente imposible para un autómata finito.

Ejemplo 2 - Balanceo y anidación sintáctica en compiladores

En el desarrollo de software y construcción de intérpretes, los AP son el fundamento algorítmico del análisis sintáctico. Un caso práctico es la validación de delimitadores de

alcance, como paréntesis `( )`, corchetes `[ ]` y llaves `{ }`, en el código fuente de un lenguaje L.

- **Proceso analítico:** Considere la evaluación de una estructura condicional anidada como `if (x == array[i]) { print(x); }`. El analizador lee la secuencia de tokens de izquierda a derecha. Cada vez que detecta un token de apertura (`(`, `[`, `{`), lo introduce en la pila. Al encontrar un token de cierre, el AP consulta el símbolo en el tope de la pila. Si el símbolo de cierre coincide matemáticamente con la naturaleza del símbolo de apertura en el tope (por ejemplo, un `]` cierra un `[`), lo desapila y continúa.
- **Control de errores:** Si el símbolo entrante no corresponde al del tope, o si al finalizar el archivo la pila no está vacía (indicando que quedó un bloque sin cerrar), el compilador emite un "Error de Sintaxis" (Syntax Error). Este principio es la base de los analizadores descendentes (LL) y ascendentes (LR) modernos.

Utilidad e Importancia (AP vs. AFD y AFND)

La limitación arquitectónica inherente de los Autómatas Finitos (tanto Determinísticos como No Determinísticos) radica en su dependencia exclusiva de un conjunto estático y finito de estados computacionales. Esto se traduce en una "amnesia algorítmica": los AFD y AFND no poseen memoria histórica más allá de su estado actual, por lo que están estrictamente limitados a reconocer **Lenguajes Regulares** (Tipo 3 en la jerarquía de Chomsky). Por esta razón, un AFD es la herramienta perfecta y suficiente para la etapa de análisis léxico (como se demostró mediante expresiones regulares y FLEX al identificar lexemas aislados). Sin embargo, resultan obsoletos para procesar jerarquías, bloques anidados o dependencias a larga distancia, ya que no pueden "recordar" cuántas veces ocurrió un evento anterior.

En contraste, el Autómata de Pila (AP) rompe esta barrera estructural mediante la incorporación de su memoria auxiliar LIFO, dotando a la máquina teórica de una capacidad de almacenamiento técnicamente infinita. Este incremento monumental en su poder de cómputo es lo que faculta a los AP para reconocer y validar **Lenguajes de Libre Contexto** (Tipo 2). En la arquitectura de un compilador, esta evolución es indispensable: mientras que el autómata finito lee las "palabras" (tokens), es el Autómata de Pila el responsable de entender la "gramática" y construir el Árbol de Sintaxis Abstracta (AST), actuando como el motor intelectual del analizador sintáctico (Parser) y garantizando que el programa fuente posea una lógica estructural coherente antes de su traducción a código máquina.

Analizador Léxico de Archivos Docker

El análisis léxico constituye la primera fase esencial en la arquitectura de un compilador o intérprete, encargada de transformar un flujo unidimensional de caracteres en una secuencia ordenada de unidades sintácticas con significado, denominadas tokens. En este apartado se documenta el diseño y la implementación de un analizador léxico (lexer) desarrollado específicamente para la validación estructural de archivos de configuración de contenedores (Dockerfiles).

Fundamentos Teóricos Aplicados

Para la concepción de esta herramienta, se parametrizan los conceptos fundamentales de la teoría de lenguajes formales:

- **Token:** Representa una categoría abstracta o clase de componente léxico dentro del lenguaje formal, tal como palabras reservadas, identificadores o símbolos operadores.

- **Lexema:** Constituye la instancia concreta de caracteres o cadena de texto que coincide con el patrón definido para un token determinado en el programa fuente.
- **Patrón:** La regla formal, descrita mediante expresiones regulares, que dictamina la validez de los caracteres evaluados.

Desde la perspectiva de la jerarquía de Chomsky, los componentes del archivo de configuración pertenecen a la categoría de lenguajes regulares. Por consiguiente, su reconocimiento computacional puede ser completamente resuelto mediante el modelado de Autómatas Finitos Determinísticos (AFD), mapeados en código mediante un motor de búsqueda iterativo que evalúa las transiciones de estado a partir de los caracteres leídos

Especificación Sintáctica del subconjunto del Lenguaje Docker

El diseño del analizador requiere establecer de forma unívoca la gramática regular que gobernará el reconocimiento del archivo. Es importante destacar que la evaluación de los patrones sigue un orden estricto de precedencia para evitar colisiones (por ejemplo, evaluar números antes que rutas). A continuación, se detalla la matriz de especificación léxica final con los patrones Regex correspondientes a cada componente sintáctico:

Componente Léxico (Token)	Descripción Operativa	Expresión Regular (Regex)
INSTRUCTION	Directivas base y palabras reservadas del ciclo de construcción en Docker.	<code>r'\b (FROM RUN CMD COPY EXPOSE WORKDIR ENV) \b'</code>
IMAGE_NAME	Identificadores de imágenes base que incluyen etiquetas de versión o variantes del SO.	<code>r'[a-zA-Z0-9_-]+:[a-zA-Z0-9_-]+'</code>
NUMBER	Parámetros numéricos enteros, típicamente asignados a la declaración de puertos de red.	<code>r'\d+'</code>

PATH	Direccionamiento de rutas absolutas o relativas en el sistema de archivos del contenedor.	<code>r'[\./]?[a-zA-Z0-9_/-]+'</code>
EQUALS	Operador de asignación explícita de valores a variables de entorno.	<code>r'='</code>
STRING	Cadenas de texto literales delimitadas por comillas dobles.	<code>r'"[^"]*"'</code>
COMMENT	Líneas destinadas a documentación interna, identificadas por un prefijo numeral (#).	<code>r'#.*'</code>
NEWLINE	Carácter de control de salto de línea, crítico para la geolocalización de coordenadas del error.	<code>r'\n'</code>
SKIP	Espacios en blanco y tabulaciones horizontales empleados como delimitadores sintácticos.	<code>r'[\t]+'</code>
MISMATCH	Patrón comodín residual diseñado para capturar cualquier elemento fuera del alfabeto del lenguaje.	<code>r'.'</code>

Arquitectura y Diseño Técnico del Motor Léxico

La solución de software construida adopta una arquitectura basada en la compilación y agregación de grupos con nombre (named capturing groups). La estructura operativa del lexer se divide en dos funciones acopladas:

- 1. Lector de flujo de caracteres:** Lee el archivo fuente en su totalidad y utiliza la rutina `re.finditer` para realizar un escaneo lineal no bloqueante del texto completo, garantizando una complejidad temporal óptima de $O(n)$ respecto al volumen de caracteres de entrada.

2. Verificador de patrones y despachador de estados: A medida que se identifican coincidencias, el motor discrimina la naturaleza del token:

- Si se detectan tokens de tipo `SKIP` o `COMMENT`, el sistema realiza un salto inmediato en el flujo sin generar salidas hacia las fases subsecuentes del compilador.
- Si se procesa un token `NEWLINE`, se incrementa el contador de líneas y se actualiza la posición absoluta del cursor para reiniciar el cálculo de la columna.
- Si el token corresponde a un `MISMATCH`, se rompe el flujo de ejecución normal emitiendo una excepción detallada de tipo léxico que precisa la coordenada exacta (línea y columna) del fallo, emulando la teoría de los estados fallidos para la gestión de errores del compilador.
- Para cualquier otro patrón válido, el analizador despacha una estructura de datos tuplada conteniendo el identificador del token, el lexema capturado y su ubicación espacial.

Implementación y Ejecución

Para el desarrollo del analizador léxico de Docker, se empleó el lenguaje de programación Python (versión 3.x), aprovechando su biblioteca estándar `re` para el procesamiento nativo de expresiones regulares, lo que elimina la dependencia de librerías de terceros. Como entorno de desarrollo integrado (IDE) se utilizó Visual Studio Code y el control de versiones se gestionó a través de Git. Para la generación del archivo ejecutable independiente solicitado, se implementó la herramienta PyInstaller.

El analizador léxico no requiere una instalación compleja por parte del usuario final, ya que se provee un archivo ejecutable precompilado (`lexer_docker.exe` / `lexer_docker`). Sin embargo, para su ejecución desde el código fuente, es necesario contar con un entorno de Python 3.x instalado. No se requieren dependencias externas.

Instrucciones de Ejecución:

1. Ubicar el archivo ejecutable o el script `lexer_docker.py` en el mismo directorio donde se encuentran los archivos `Dockerfile` a evaluar.
2. Abrir la interfaz de línea de comandos (Terminal o Símbolo del sistema).
3. Ejecutar el analizador pasando como argumento el nombre del archivo fuente.
 - Ejecución vía código fuente: `python lexer_docker.py nombre_del_archivo`
 - Ejecución vía binario: `./lexer_docker nombre_del_archivo`

```
--- Analisis completado con éxito ---
PS C:\Users\User\Documents\Lenguajes-Compiladores-Kernel14\TEMA4\Codigo\Asignacion 2> python
lexer_docker.py Dockerfile_basico
--- Analizando archivo: Dockerfile_basico ---
('INSTRUCTION', 'FROM', 2, 0)
('IMAGE_NAME', 'ubuntu:latest', 2, 5)
('INSTRUCTION', 'WORKDIR', 3, 0)
('PATH', '/app', 3, 8)
('INSTRUCTION', 'COPY', 4, 0)
('PATH', '.', 4, 5)
('PATH', '.', 4, 7)
('INSTRUCTION', 'RUN', 5, 0)
('PATH', 'make', 5, 4)
--- Analisis completado con éxito ---
PS C:\Users\User\Documents\Lenguajes-Compiladores-Kernel14\TEMA4\Codigo\Asignacion 2> python
lexer_docker.py Dockerfile_web
--- Analizando archivo: Dockerfile_web ---
('INSTRUCTION', 'FROM', 1, 0)
('IMAGE_NAME', 'node:18-alpine', 1, 5)
('INSTRUCTION', 'ENV', 2, 0)
('PATH', 'PORT', 2, 4)
('EQUALS', '=', 2, 8)
('NUMBER', '8080', 2, 9)
('INSTRUCTION', 'WORKDIR', 3, 0)
('PATH', '/src', 3, 8)
('INSTRUCTION', 'COPY', 4, 0)
('PATH', './package.json', 4, 5)
('PATH', './', 4, 20)
('INSTRUCTION', 'RUN', 5, 0)
('PATH', 'npm', 5, 4)
('PATH', 'install', 5, 8)
('INSTRUCTION', 'EXPOSE', 6, 0)
('NUMBER', '8080', 6, 7)
('INSTRUCTION', 'CMD', 7, 0)
('STRING', 'npm start', 7, 4)
--- Analisis completado con éxito ---
PS C:\Users\User\Documents\Lenguajes-Compiladores-Kernel14\TEMA4\Codigo\Asignacion 2> python
lexer_docker.py Dockerfile_error
--- Analizando archivo: Dockerfile_error ---
('INSTRUCTION', 'FROM', 1, 0)
('IMAGE_NAME', 'python:3.9', 1, 5)
('INSTRUCTION', 'COPY', 3, 0)
ERROR LÉXICO: '?' inesperado en la línea 3, columna 5
❖ PS C:\Users\User\Documents\Lenguajes-Compiladores-Kernel14\TEMA4\Codigo\Asignacion 2>
```

Analizador Léxico para el Lenguaje Rust

La presente sección aborda el diseño de un analizador léxico automatizado para un subconjunto del lenguaje de programación Rust. A diferencia del enfoque implementado en la Actividad 2, donde el motor de análisis se construyó de manera manual ("desde cero"), en este apartado se utiliza la herramienta de metacompilación FLEX para la generación automática del código en lenguaje C.

Manual de Usuario del Metacompilador FLEX

FLEX (Fast Lexical Analyzer Generator) es una herramienta de software que facilita la creación rápida de analizadores léxicos. Su función principal es tomar como entrada un archivo de especificación de texto (con extensión `.l`) que contiene un conjunto de expresiones regulares y transformarlo en un código fuente en lenguaje C (`lex.yy.c`). Este archivo generado en C contiene un autómata finito determinístico (AFD) altamente optimizado, capaz de escanear flujos de caracteres y ejecutar acciones específicas cuando reconoce un patrón.

Estructura del archivo de especificación (.l): El archivo fuente que FLEX lee está estrictamente dividido en tres secciones, separadas por el delimitador `%%`:

- 1. Sección de Definiciones (`%{ ... %}`):** Ubicada al inicio del archivo. Aquí se declaran las bibliotecas de C necesarias (como `<stdio.h>`) y las opciones del compilador (como `%option noyywrap` para indicar que se leerá un solo archivo a la vez).
- 2. Sección de Reglas (`%% ... %%`):** Es el núcleo del analizador. En el lado izquierdo se definen las expresiones regulares (los patrones) y en el lado derecho, encerrado entre

llaves `{ }`, se escribe el código en C que se ejecutará al encontrar una coincidencia (típicamente, imprimir el token y su lexema asociado mediante `yytext`).

- 3. Sección de Código de Usuario:** Ubicada al final del archivo. Contiene funciones en C que serán copiadas literalmente al analizador final, siendo la principal la función `main()`, encargada de abrir el archivo fuente (`yyin = file;`) y hacer el llamado inicial a la función de lectura léxica (`yylex();`).

Descripción del Lenguaje L (Subconjunto de Rust)

El lenguaje Rust posee una gramática rica y compleja. Para los fines de este proyecto, se ha delimitado un "Lenguaje L" que representa un subconjunto funcional de Rust, enfocado estrictamente en la declaración de variables y la ejecución de la macro principal de salida por consola.

La delimitación de este subconjunto permite construir un autómata finito capaz de reconocer estructuras clásicas como el siguiente ejemplo de "Hola Mundo":

```
// Programa principal
fn main() {
    let mut numero: i32 = 10;
    println!("Hola");
}
```

Matriz de Componentes Léxicos (Tokens) Diseñados: A continuación, se define la tabla de tokenización que será traducida a las reglas de FLEX para el lenguaje L:

Token Asignado	Descripción del Componente	Expresión Regular en FLEX
----------------	----------------------------	---------------------------

TK_FN	Palabra reservada para la declaración de funciones.	<code>fn</code>
TK_LET	Palabra reservada para instanciar variables.	<code>let</code>
TK_MUT	Modificador para permitir mutabilidad en variables.	<code>mut</code>
TK_I32	Palabra reservada para el tipo de dato entero de 32 bits.	<code>i32</code>
TK_PRINTLN	Invocación de la macro estándar de impresión en consola.	<code>println!</code>
TK_IDENTIFICADOR	Nombres asignados a variables o métodos (inician con letra).	<code>[a-zA-Z_][a-zA-Z0-9_]*</code>
TK_NUMERO	Valores numéricos enteros de cualquier longitud.	<code>[0-9]+</code>
TK_CADENA	Texto literal delimitado por comillas dobles.	<code>"[^"]*"</code>
TK_SIMBOLO	Caracteres sintácticos estructurales del subconjunto.	<code>{ } () : ; =</code>
TK_COMENTARIO	Líneas de documentación interna.	<code>//.*</code>

```

@sccl-r → .../Lenguajes-Compiladores-Kernel4/TEMA4/Codigo/Asignacion 3
● (main) $ ./lexer_rust prueba.rs
TOKEN: TK_FN, Lexema: fn
TOKEN: TK_IDENTIFICADOR, Lexema: main
TOKEN: TK_SIMBOLO, Lexema: (
TOKEN: TK_SIMBOLO, Lexema: )
TOKEN: TK_SIMBOLO, Lexema: {
TOKEN: TK_LET, Lexema: let
TOKEN: TK_MUT, Lexema: mut
TOKEN: TK_IDENTIFICADOR, Lexema: numero
TOKEN: TK_SIMBOLO, Lexema: :
TOKEN: TK_I32, Lexema: i32
TOKEN: TK_SIMBOLO, Lexema: =
TOKEN: TK_NUMERO, Lexema: 10
TOKEN: TK_SIMBOLO, Lexema: ;
TOKEN: TK_PRINTLN, Lexema: println!
TOKEN: TK_SIMBOLO, Lexema: (
TOKEN: TK_CADENA, Lexema: "Hola"
TOKEN: TK_SIMBOLO, Lexema: )
TOKEN: TK_SIMBOLO, Lexema: ;
TOKEN: TK_SIMBOLO, Lexema: }
@sccl-r → .../Lenguajes-Compiladores-Kernel4/TEMA4/Codigo/Asignacion 3
○ (main) $ █

```

FLEX en Seguridad Informática

En el ámbito de la seguridad informática, la validación de configuraciones y políticas de red es una tarea crítica. Los firewalls, los sistemas SIEM (Gestión de Eventos e Información de Seguridad) y los sistemas de detección de intrusos (IDS) operan mediante la ingesta constante de reglas definidas por analistas. En cumplimiento con la investigación requerida para este tema, la presente sección explora cómo la teoría de lenguajes formales y los metacompiladores pueden integrarse en este dominio para optimizar la tokenización y validación de firmas de seguridad.

Esto nos lleva a la selección del lenguaje, el cual es Reglas de detección de **Snort** (Sistema de Prevención e Intrusión de Red - NIDS).

Justificación de selección: Snort utiliza un lenguaje formal declarativo, estructurado y altamente determinístico para definir "firmas" de tráfico de red malicioso. Una regla típica de Snort se compone de una cabecera lógica (que define la acción, el protocolo, el origen, el destino y el flujo) y un cuerpo de opciones (que especifica los patrones del ataque). Dado que obedece a una gramática estricta, sus componentes se comportan funcionalmente como un lenguaje regular, haciéndolo un candidato ideal para el análisis lexicográfico.

La Intervención de FLEX

En los entornos de Operaciones de Seguridad (SOC), los ingenieros y analistas redactan y actualizan miles de firmas de detección de manera constante. Este proceso de creación manual introduce un margen significativo de error humano; un simple fallo léxico (como escribir `udP` en lugar de `udp`, o la omisión de un delimitador de cadena) puede provocar que el motor de inspección de paquetes del IDS falle al compilar la regla, dejando la

red vulnerable o, en el peor de los casos, consumiendo excesivos ciclos de CPU intentando interpretar una sintaxis ambigua.

Es en este punto donde la integración de un metacompilador como FLEX aporta un valor estructural invaluable. FLEX permite automatizar la generación de un analizador léxico en lenguaje C (altamente eficiente a nivel computacional) que actúe como un pre-procesador de validación en la canalización (pipeline) de seguridad. Antes de que las reglas sean empujadas al entorno de producción, el lexer de FLEX procesaría los archivos de firmas en cuestión de milisegundos, tokenizando estrictamente las palabras reservadas, validando el formato de las direcciones IP y capturando cualquier carácter ilícito (estado de error). Esto garantiza que el motor del firewall o IDS reciba únicamente instrucciones léxicamente inmaculadas, optimizando el rendimiento de la red y evitando caídas del servicio por reglas mal formadas.

Diseño Específico de la Tokenización para Firmas Snort

Para materializar esta solución, se define a continuación la matriz de tokenización que alimentaría el archivo de especificación `.l` de FLEX. El modelo abstrae la cabecera estándar de una regla Snort (ejemplo: `alert tcp $EXTERNAL_NET any -> $HTTP_SERVERS 80 (msg:"Ataque";)`):

Descripción / Uso Sintáctico	Token Propuesto	Expresión Regular en FLEX
Acciones del motor: Define la directiva de mitigación o registro del IDS.	TK_ACTION	<code>alert log pass drop sdrops</code>
Protocolo de red: Delimita la capa de transporte/red a inspeccionar.	TK_PROTOCOL	<code>tcp udp icmp ip</code>

Variables de entorno: Macros de red definidas en la configuración global.	TK_VAR_NET	<code>\\$[a-zA-Z_][a-zA-Z0-9_]*</code>
Direccionamiento IP: Formato estricto para direcciones IPv4 origen/destino.	TK_IPV4	<code>([0-9]{1,3}\.){3}[0-9]{1,3}</code>
Puertos de conexión: Puertos numéricos específicos o el comodín general.	TK_PORT	<code>[0-9]+ any</code>
Operador de direccionalidad: Establece el vector de flujo del tráfico.	TK_DIR_OP	<code>-> <- <></code>
Cadenas de alerta (Metadata): Mensajes descriptivos encapsulados en comillas.	TK_MSG_BODY	<code>msg:"[^"]*"*</code>
Delimitadores estructurales: Caracteres obligatorios para el cuerpo de la regla.	TK_DELIMITER	<code>[() ;]</code>

Una vez que el archivo de firmas de seguridad es procesado exitosamente por este analizador léxico, el flujo unidimensional de texto se convierte en una estructura de tokens completamente higienizada. Esta salida está lista para ser consumida de forma segura por el analizador sintáctico del sistema IDS, el cual se encargará finalmente de validar que el orden lógico de los componentes cumpla con la gramática de Snort, demostrando así la escalabilidad de las herramientas teóricas de compilación en el mundo real de la ciberseguridad.

Conclusiones

El desarrollo de las actividades expuestas en este informe evidencia la importancia fundamental del análisis léxico como mecanismo de control y estructuración de datos en las ciencias de la computación. A través de la implementación práctica, se demostró que los Autómatas Finitos Determinísticos, modelados mediante expresiones regulares, son herramientas altamente eficientes y versátiles para la validación de lenguajes regulares, tal como se comprobó en la construcción del analizador de archivos Docker.

Sin embargo, el proyecto también corroboró que, ante la complejidad escalar de lenguajes de programación robustos como Rust, el uso de herramientas de metacompilación como FLEX resulta indispensable. Este software no solo reduce drásticamente los tiempos de desarrollo de la fase léxica, sino que minimiza la introducción de errores humanos al generar automáticamente un código fuente en C altamente optimizado.

La integración del estudio teórico de los Autómatas de Pila (AP) permitió comprender los límites arquitectónicos del análisis léxico, evidenciando que, si bien los autómatas finitos son excepcionales para tokenizar palabras, se requiere la memoria dinámica LIFO de los AP para la validación de gramáticas de libre contexto en las etapas sintácticas posteriores. Finalmente, la reflexión aplicada a los sistemas de detección de intrusos (Snort) demostró que el dominio de estas herramientas trasciende el diseño de compiladores académicos, consolidándose como una competencia técnica vital para la automatización de infraestructuras, el procesamiento seguro de configuraciones y el fortalecimiento proactivo de la ciberseguridad.